



DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001-2015 Certified Institution)

Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru - 560 111. India

DEPARTMENT	ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING				
Semester: VII	Course Title: Data Security & Privacy Lab			Course Code: 22AI73	
PROGRAM QUESTION	Hash Functions and Obfuscation:a)Implement common hash functions (e.g., SHA-256, MD5) using Python's hashlib. Input: A string or file. Output: Its hash value.				
STUDENT NAME	Udayaraj		USN	1DS22AI057	
LAB MARKS	Program Execution (10)	Source Code (10)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	Cursor				
MAP COURSE OUTCOME	CO1				
MAP PROGRAM OUTCOME	PO2, PO3, PO5, PO9, PO11				
MAP PROGRAM SPECIFIC OUTCOME	PSO1, PSO2, PSO3				
UML / SEQUENCE DIGRAM	<pre>sequenceDiagram actor User participant Hash1 as Hash participant Obfuscator1 as "Obfuscator" participant Hash2 as Hash participant Obfuscator2 as Obfuscator User->>Hash1: Submit input (string/file) & select hash type activate Hash1 Hash1->>Obfuscator1: Compute hash (SHA-256, MD5, etc.) activate Obfuscator1 Obfuscator1-->>Hash2: Return hash value deactivate Obfuscator1 Hash2-->>User: deactivate Hash2 User->>Obfuscator2: Submit Python function for obfuscation activate Obfuscator2 Obfuscator2->>Obfuscator2: Apply obfuscation techniques activate Obfuscator2 Obfuscator2-->>Hash2: Return obfuscated code deactivate Obfuscator2 Hash2-->>User: </pre>				
COURSE COORDINATOR	Dr ARUNA M G & Prof ENSTEIH SILVIA				

Code for Program 7:

7) Hash Functions and Obfuscation:a)Implement common hash functions (e.g., SHA-256, MD5) using Python's hashlib.
Input: A string or file. Output: Its hash value.

b) Explore obfuscation techniques

```
import streamlit as st, hashlib, base64

def hash_text(text, algo): return hashlib.new(algo, text.encode()).hexdigest()
def hash_file(file, algo): return hashlib.new(algo, file.read()).hexdigest()
def obfuscate_code(code):
    encoded = base64.b64encode(code.encode()).decode()
    return f'import base64\nexec(base64.b64decode("{encoded}").decode())'

st.title("🔒 Hash & Code Obfuscator")
choice = st.sidebar.radio("Select Mode", ["Hash Generator", "Code Obfuscator"])
if choice == "Hash Generator":
    input_type = st.radio("Input type", ["Text", "File"])
    if input_type == "Text":
        txt = st.text_area("Enter text")
        if st.button("Generate Hash") and txt:
            st.write("MD5:", hash_text(txt, "md5"))
            st.write("SHA-256:", hash_text(txt, "sha256"))
        else:
            f = st.file_uploader("Upload file")
            if f and st.button("Generate File Hash"):
                st.write("MD5:", hash_file(f, "md5"))
                f.seek(0)
                st.write("SHA-256:", hash_file(f, "sha256"))
    else: # Code Obfuscator
        code = st.text_area("Paste Python code")
        if st.button("Obfuscate") and code.strip():
            st.subheader("🔍 Original Code"); st.code(code, language="python")
            st.subheader("🛡️ Obfuscated Code"); st.code(obfuscate_code(code), language="python")
```

RESULT:

